MEDHAVA

Medhava BOS — how to build it

Start here if you have just downloaded `MEDHAVA_BOS.zip`. This is the ordered path from that file to a running website, and then the loop you repeat once per app.

6 parts, 36 steps. Every command below was checked against the repository before this document was written: an `npm run` names a script that exists, and a `node` command names a file that is really there.

What you are starting from

	Count	State
Modules	22	specified · a navigation page each
Apps	**113**	**2 built** — Inventory (stock on hand, receipts) and Sales (recording a sale end to end)
Database tables	151	built, running, isolated
Rules	285	**88 enforced by a test that runs**; the rest are the queue

The platform underneath is real and finished: the schema executes into PostgreSQL, row-level security is enforced by the database rather than by application code, sessions carry a tenant and a company, and no business query can reach the data without both. What remains is the apps — 111 of them.

> *Every module page carries its real app names with an on-screen mark saying the screens are specified and not built.* **Leave that mark until the app is genuinely built.** *A list of app names on a working shell reads as a working app.*

Contents

5. **When something breaks** — 7 steps
6. **Going live** — 3 steps

Part 1 · Get it running

About thirty minutes, most of it waiting for `npm ci`. Do not skip the verify step — building on a suite you have not seen pass is how a whole day goes into a fault that was there before you started.

1.1 · Check the machine has Node 22.5 or newer

The core tests use `node:sqlite`, a built-in that landed in 22.5. On Node 20 they fail with "No such built-in module: node:sqlite" — 32 of them, all pointing at the wrong thing.

```
node --version
```

You should see — v22.5.0 or higher. If it is lower, install a newer Node before going on.

Know it worked — The number after v is 22.5 or more.

1.2 · Unpack the archive and go into it

```
unzip MEDHAVA_BOS.zip
cd medhava-bos
```

You should see — A folder holding CLAUDE.md, START_HERE.md, medhava/, core/, brand/ and package.json.

Know it worked — START_HERE.md is there. Read it — it is one page and it says what is built and what is not.

1.3 · Install the toolchain from the committed lockfile

From the lockfile, not from the registry's latest: the versions are pinned so your machine gets the same ones this was verified against.

```
npm ci
```

You should see — A few hundred packages, no errors. It takes a couple of minutes.

Know it worked — node_modules/ exists and the command exited without an error.

1.4 · Prove the copy you have actually works, before touching anything

This is the whole point of the step. Everything after it assumes a green starting point, and a fault you inherit is indistinguishable from one you cause.

```
npm run test:product
```

You should see — Each gate reports in turn, then the suites: the schema in real PostgreSQL, the sales module, and the browser checks. The command exits 0.

Know it worked — Exit code 0. Check it explicitly if your shell does not show it: `echo $?`.

> ⚠️ *If it says "7 browser checks SKIPPED, not passed", your machine has no Chromium. That is step 1.5 and the run still exits 0 — but those checks did not run, so the screens are unverified until they do.*

1.5 · Install Chromium, if the browser checks were skipped

playwright-core is in the lockfile; the browser it drives is not, because that is a few hundred megabytes of platform binary no lockfile should carry.

```
npx playwright install chromium
```

You should see — A download, then `npm run test:product` reports 9 browser checks passing instead of skipping.

Know it worked — Run `npm run test:product` again. The SKIPPED banner is gone.

1.6 · Start it

```
npm start
```

You should see — The schema loads into PostgreSQL in a few seconds, then:

PostgreSQL 18.3 (PGlite) on wasm32-unknown-linux-gnu 151 tables · 135 row-level policies active 2 businesses · 3 companies (seeded) open http://localhost:4000

Know it worked — The URL answers. Leave it running and open it in a browser.

> ⚠️ *No database to install. PGlite is real PostgreSQL compiled to WebAssembly and it is a dependency, not a service. In production you swap this one file for a connection pool — see DEPLOYMENT.md — and nothing above it changes.*

1.7 · Sign in and open the Isolation page

It is the platform's central claim made visible: two unrelated businesses on one database, neither able to reach the other.

You should see — Sign in as owner@anjali.demo. The page shows, for orders, products and channels, what this company can see against what the database actually holds — 3 against 7. The gap is enforced by PostgreSQL row-level security, not by a filter the code remembered.

Know it worked — Change the company in the top-right selector. The figures change and never overlap.

1.8 · Record a sale, so you have seen the one built write path work

Reading is not the same as watching it post. This is the pattern every other app will follow.

You should see — Choose a channel, add a line, press "Post the sale". A receipt names the order and the invoice, and shows the tax split to the paisa. One transaction moved the stock, raised the invoice and posted the ledger — or none of it would have happened.

Know it worked — Open Orders. The sale is there. Open Isolation. The other company still cannot see it.

Part 2 · Read these five things before writing any code

In this order. Together they are about an hour, and they replace a week of finding out the same things by breaking them.

2.1 · CLAUDE.md — the working agreement

It is loaded automatically at the start of every Claude Code session, so it governs the agent whether or not you have read it. Section 0 is the one that matters most: Medhava is the product, a tenant is a customer, and the two are never mixed.

Know it worked — You can say what "derive, never retype" means and why it is treated as fabrication to break it.

2.2 · brand/site/modules.js — the one canonical list

Every module and app name, in order. Nothing types a count from it; everything reads it. It has already changed twice, which is why.

```
node -e "const M=require('./brand/site/modules.js');console.log(M.length+' modules, '+M.reduce((n,m
```

You should see — The module list with its app counts, derived from the file rather than typed here.

2.3 · brand/site/rules.js — what each module must do, and must never do

Every rule carries a `never`: the wrong behaviour it exists to prevent. A rule marked ENFORCED must name a file and a test that really exist, and checkrules.js fails the build otherwise — so a rule cannot claim a proof it does not have.

```
node brand/site/checkrules.js --summary
```

You should see — A table of rules per module, how many are enforced, and the honest total: the enforced ones are backed by a test that runs today; the rest are the build queue.

2.4 · medhava/ — the running platform, six files

This is the whole product as it stands. It is small on purpose.

You should see — server/db.js — the isolation everything rests on, and the only way to the data. server/api.js — every business route wrapped in guard(). server/inventory.js — the one stock number, module 03. Derived from movements, never stored. server/sales.js — a sale, module 05, which issues stock THROUGH module 03 so a sale of more than exists is refused whole. seed/demo.js — two unlike businesses. web/app.js — the screens.

Know it worked — You can explain why withContext() drops to the `authenticated` role, and what would happen if it did not.

> ⚠️ *Read the comment at the top of db.js in full. It records a measurement: a superuser bypasses every row-level policy even with FORCE ROW LEVEL SECURITY, so an app that connects as itself has no isolation and nothing about the running system looks wrong.*

2.5 · medhava/test/sales.test.js — the worked example of the test discipline

This is the file to imitate. Every check records what was planted to make it fail and what it said when it caught it. Three of its checks were found to be decoration that way, including one that restated its own arithmetic and could never fail.

```
node medhava/test/sales.test.js
```

You should see — 10 checks, each naming the rule it enforces. The one to read first is S7 — if the ledger refuses, the stock never moved.

Part 3 · Build the next app — the loop, repeated once per app

One app at a time, always in this order. The order is not a preference: writing the test after the code produces a test shaped like the code rather than like the rule, and it will pass on the bug.

3.1 · Pick one app, and read the rules it must satisfy

The rules are the specification. Building from the app name alone invents requirements and misses the ones that matter.

```
node -e "const R=require('./brand/site/rules.js');R.filter(r=>r.mod==='03').forEach(r=>console.log(
```

You should see — Every rule for module 03, with what it must never do instead. Change the two digits for a different module.

Know it worked — You have a list of the rules you intend to enforce, and know which ones you are deliberately leaving SPECIFIED.

3.2 · Write the server module, next to sales.js

Follow the existing pattern rather than inventing one. It takes a scope from the session and reaches the database only through db.withContext or db.withTransaction.

Know it worked — Nothing in the file takes a company id from the request body. The company comes from the session, always.

> ⚠️ *If the operation writes more than one table, it MUST be withTransaction. A sale that deducts stock and then fails to post the ledger leaves the goods gone and the books untouched, and nothing about the running system looks wrong — the stock figure is simply short forever, with no record to reconcile it against.*

3.3 · Add the route, wrapped in guard()

guard() gives 401 with no session, 409 with no company chosen, 403 for a company you do not belong to. There is deliberately no route that opts out.

Know it worked — curl the route with no cookie. It answers 401, not an empty list.

> ⚠️ *A business route that returns `200 []` to an unauthenticated caller reads as success in every log anybody will look at, and is exactly what a system with broken isolation returns.*

3.4 · Add the screen

A route with no screen is not an app. Follow the shape of the "Record a sale" screen: a refusal must name the rule that refused it, because "invalid input" teaches nobody what to change.

Know it worked — Drive it in a browser, not with curl. A form that posts correctly to curl and does nothing to a click is a form nobody can use — that exact defect shipped here once, and every API test was green while it did.

3.5 · Write the test, one check per rule you enforced

Name the rule in the check’s title. Six months from now the connection between a rule and the thing that proves it is the only thing that keeps either honest.

Know it worked — Every check’s failure message says what went wrong in business terms, not "expected 3 to equal 4".

3.6 · Prove every check fails before you trust that it passes

THE STEP PEOPLE SKIP, AND THE ONLY ONE THAT MAKES THE REST MEAN ANYTHING. Break the thing the check is supposed to catch, run the suite, and confirm that check — not some other one — goes red. Then put it back.

You should see — A check that catches nothing is decoration. A check that catches the wrong thing is measuring something else. Both look identical to a green run.

Know it worked — For each check: it went red, it was the right one, and its message named the real problem.

> ⚠️ *When a plant does not fire, suspect the plant before the check. Two of the plants written here were aimed at code that did not hold the value being tested, and the checks were right to stay green.*

3.7 · Promote the rules you enforced, in brand/site/rules.js

Change `...S` to `state:'ENFORCED'` and add `by:` naming the file and the exact test title. This is what makes the rulebook a record rather than a wish.

```
node brand/site/checkrules.js --summary
```

You should see — The enforced count goes up. If you named a test that does not exist, this refuses — it reads the file and looks for the title.

3.8 · Run everything, and read the exit code

```
npm run test:product
```

You should see — Exit 0, with your new checks in the list.

Know it worked — Exit code 0. Not "it looked fine".

> ⚠️ *Run this AFTER your last edit, not before. A file created after the suite ran is untested — that happened here, and CI caught what the local run could not have.*

3.9 · Commit, saying what you verified

The message is where the next person learns what was proven and what was assumed.

```
git add -A
git commit
```

Know it worked — The message names the rules enforced, what was planted to prove each check, and the command whose output you are relying on.

Part 4 · Doing it with Claude Code

The archive is set up so an agent starts with the right context instead of guessing.

4.1 · Open the unpacked folder as the project

CLAUDE.md is loaded automatically from the project root. Opening a parent folder or a subfolder means it is not, and the agent works without the rules.

Know it worked — Ask it what CLAUDE.md section 0 says. If it cannot answer, the folder is wrong.

4.2 · Paste MEDHAVA_BOS_PROMPT.md as the first message

It says what already exists, so the agent does not rebuild a 151-table schema that runs. Its first section verifies you have the right thing and stops if you do not — that check exists because an archive of documents was once handed to an agent, which then invented the files it could not find.

Know it worked — Its opening check passes. If it reports missing paths, you are in the wrong folder or have an incomplete copy.

4.3 · Ask for one app at a time, and name the module number

"Build the CRM module" is a week of work with no checkpoint. "Build module 03 Inventory, the on-hand figure only, with a test per rule proven red first" is a day with something to verify at the end of it.

Know it worked — The ask names a module number, one deliverable, and the command that will decide whether it worked. If you cannot say what you would run to check it, the ask is too big.

> ⚠️ *Do not accept "tests pass" without the output. The anti-cheat skill in .claude/skills/ is installed for this and applies automatically — but you are the one who has to notice when a claim arrives without evidence attached.*

4.4 · Check the work yourself in the browser before moving on

Every screen defect found in this project was found by driving it in a browser, and every one of them was invisible to the API tests that were green at the time.

```
npm start
```

Know it worked — You clicked the thing. It did what it said.

Part 5 · When something breaks

Every one of these happened during the build. The message is quoted as it actually appears.

5.1 · "No such built-in module: node:sqlite"

You should see — Node is older than 22.5. 32 of the core tests fail together and none of them says the word "node". Install a newer Node.

5.2 · "7 browser checks SKIPPED, not passed"

You should see — No Chromium on the machine. `npx playwright install chromium`. The run still exits 0 and that is deliberate — a missing browser is your environment, not a defect in the code — but the screens are unverified until those checks run.

5.3 · "Port 4000 is already in use"

You should see — Something is already listening, often an earlier `npm start` you forgot. Start on another port: `PORT=4100 npm start`.

5.4 · "refusing to query without both a tenant and a company"

You should see — Your code reached the database without a scope. This is db.js refusing on purpose rather than returning rows. Pass the scope from the session — never from the request body.

5.5 · "…is not in the archive" from mkprompts or mkskills

You should see — A document names a path that does not exist. Either the path is wrong or the file is genuinely missing. These gates exist so a reader is never sent to a file nobody wrote.

5.6 · checkcoverage: "sits with the documents and is neither delivered nor explained"

You should see — You added a markdown file at the root. Every document there owes a decision — add it to DOCS in brand/delivery/manifest.js, or to NOT_DELIVERED with a reason.

5.7 · A check that will not go red when you break the thing it tests

You should see — Suspect the plant first: it may be aimed at code that does not hold the value. If the plant is right and the check still passes, the check is decoration — rewrite it against an independent recomputation rather than against its own parts.

Part 6 · Going live

Not yet — but here is what changes when you do, so nothing in the build surprises you later.

6.1 · Swap PGlite for a PostgreSQL server

One file changes: medhava/server/db.js opens a connection pool instead of an in-process database. Everything above it is unchanged, because everything above it only ever calls withContext().

Know it worked — DEPLOYMENT.md section 6a. Read it before you provision anything.

6.2 · Connect as a role that is neither a superuser nor the owner of the tables

THE SINGLE LINE THAT DECIDES WHETHER ISOLATION EXISTS AT ALL. A superuser bypasses every policy even on a table with FORCE ROW LEVEL SECURITY. That was measured against a running database, not

assumed.

```
node core/tests/live.test.js
```

You should see — It runs the schema into a real PostgreSQL and asks it, as three different roles, what each can see. Read what it prints about the superuser.

> ⚠️ *Get this wrong and every screen still works, every report still returns numbers, and one business is reading another's books.*

6.3 · Follow DEPLOYMENT.md for the rest

nginx, the systemd unit, backups, secrets. It is a runbook rather than a discussion, and it assumes the checks above already pass.

Know it worked — Every command in it ran and you read its output.

Where to look things up

Question	File
What are the modules and apps?	`brand/site/modules.js` — the one canonical list
What must each module do?	`brand/site/rules.js` — 285 rules
What is the database?	`core/schema.postgres.sql`
Why is it shaped this way?	`MEDHAVA_ARCHITECT.md` — every decision with what would make it wrong
How does each layer work?	`MEDHAVA_BUILD_GUIDE.md`
What gets built, in order?	`MEDHAVA_PLAN_OF_ACTION.md`
How does it go live?	`DEPLOYMENT.md`
What are the rules for changing this repo?	`CLAUDE.md`
Where the spec contradicts itself	`SPEC_CONFLICTS.md` — unresolved on purpose

Every technical word this guide uses, in plain language

14 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

tenant

One business using the platform. Its people, its data and its settings are its own.

Us building mein ek office. Aapka office, aapka saamaan, aapka taala.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

row-level security

A lock inside the database itself, so one business physically cannot read another business’s records — even if the software above it has a bug.

Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.

backup

A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.

Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

On `MEDHAVA_BUILD_GUIDE.md` Part 13: it is the path from an *empty machine* to a live product — `git init`, `npm init`, creating a database by hand. That is how this platform was built, and it is still the right reference for the deployment stage. It is not the path for somebody holding the archive, where all of that already exists. Follow this document instead, and read Part 13 when you reach Part 6 below.
